

CSCE 713 SOFTWARE SECURITY · MODULE 10 · LECTURE NOTES

The Secure Development Lifecycle

Process that changes outcomes, and process that does not.

MODULE OUTCOMES

- Explain where in the lifecycle each practice belongs.
- Explain why secure defaults beat developer education.
- Design a security gate that does not get bypassed.
- Explain what to do with a vulnerability report.
- Assess a development process honestly.

1 Where each practice belongs

Stage	The practice	What it catches
Design	Threat modelling (CSCE 701 Module 01).	Design flaws — which no tool finds , because a tool does not know what the system was for. See the note.
Dependency selection	Adoption review and continuous scanning (CSCE 701 Module 08).	Inherited defects, and deserialisation gadgets (Module 06 §1).
Writing code	Safe libraries and parse-don't-validate types (Module 04 §4).	Whole classes, by construction — which is §2's argument and the highest-leverage row.
Commit	Static analysis on the diff (Module 07 §4).	Pattern-matchable defects, before merge , where the author is still thinking about the code.
Build and test	Sanitisers and fuzzing (Modules 08 and 09).	Memory safety and parsing defects — the pairing of Module 09 §3.
Review	Security review of the risky changes (Module 11).	Logic and authorisation defects , which are exactly what the tooling cannot see.
Operate	Detection and incident response (CSCE 701 Modules 09 and 10).	Everything above that got through — and some always does.

The design row is the one with no automated substitute — and it is the one most often skipped, because it produces a document rather than a passing check, which makes it invisible in every dashboard. Which is CSCE 701 Module 12's measurement problem deciding a process question: the practice with the highest value is the one hardest to show credit for.

“SHIFT LEFT” IS RIGHT ABOUT COST AND WRONG AS A STRATEGY

The cost claim is simply true: a design flaw found at design time costs a conversation, and the same flaw found after release costs a redesign, a migration, and possibly an incident. Nobody disputes this.

But 'shift left' is frequently implemented as moving work onto developers — more training, more checklists, more tools in the pipeline, more questionnaires — which spends the compliance budget (CSCE 701 Module 11 §1) and therefore reduces compliance with whatever mattered most.

And the practices that actually scale are precisely the ones that require no developer effort at all: a safe default, a type that cannot be misused, a framework that escapes automatically (§2) — none of which appear in a training completion metric.

So the honest version is: move the *guarantee* left, not the *work* — which is the distinction that separates a security programme that holds up from one that decays quietly after the first serious deadline, and it is worth stating in exactly those words when the programme is being designed.

2 Defaults over education

A FRAMEWORK THAT MAKES THE DEFECT UNEXPRESSIBLE BEATS ANY AMOUNT OF TRAINING

Training decays, turns over with staff, and competes with everything else for attention — and its measured effect on defect rates is modest, which is an uncomfortable finding that the industry has largely declined to act on (CSCE 701 Module 11 §2's parallel with phishing training).

A template engine with contextual autoescaping enabled by default eliminates the entire cross-site scripting class for every developer on the team, including the one who has never heard of it (Module 04) — and it keeps working when that developer leaves.

And the pattern generalises widely: a query builder that cannot concatenate strings into SQL, a path type that cannot escape its root directory, an HTTP client that refuses internal addresses (Module 06 §3), a serialiser that cannot name a class to construct (Module 06 §1).

Which is CSCE 701 Module 11 §3's defaults argument and CSCE 711 Module 01 §4's misuse-resistant API argument — three courses in one semester converging independently on the same conclusion, which is worth noticing and is the strongest single result in this semester.

- Find the defect class you keep fixing — which your own incident and review history already tells you — and ask what interface would make it unexpressible.
- Provide that interface, and make it the easiest path — because a safe API that is harder to use than the unsafe one loses, every time, under deadline. This is the step that decides the outcome, and it is a usability requirement rather than a security one.
- Then make the unsafe path hard to reach: a lint rule banning it, a private constructor, a deprecation warning with a documented migration, or removal from the public API surface.

- **And add a static analysis rule preventing new call sites** (Module 07 §4) — **which is what makes the migration actually finish**, rather than stalling at ninety percent with the remaining call sites permanently exempt.
- **Which converts one fix into a permanent property of the codebase** — and is the mechanism by which a security programme accumulates rather than merely maintaining. **Every defect you fix is an opportunity to do this once, and the opportunity expires when attention moves on.**

3 Gates

- **Fast. A check that adds twenty minutes to every change will be skipped**, worked around, or disabled — which is CSCE 701 Module 11 §4's friction question, and the answer is measured in seconds rather than principles.
- **Scoped to new code. Baseline the existing findings** (Module 07 §3), **so the gate never demands a cleanup project as the price of enabling it** — which is the single commonest reason a gate is never turned on.
- **Specific. A finding that names the line and the fix gets fixed; one that names a risk category gets dismissed**, because the author cannot act on it and has other work.
- **With a documented, auditable override** — because **a gate with no override gets removed entirely the first time it blocks something genuinely urgent**, and then you have neither the gate nor the override.
- **And owned**, so that false positives get fixed in the rule rather than suppressed in the code — which is the difference between a gate that improves and one that decays (Module 07 §4). **The override is counter-intuitive and essential: an override that is logged and periodically reviewed preserves the gate, and no override destroys it**, which is a general lesson about controls and people.

4 Receiving and making reports

MAKE IT EASY TO REPORT A VULNERABILITY TO YOU

Publish a security contact and a disclosure policy — a `security.txt` file, or a documented address on your site — because **a researcher who cannot find where to report will publish instead, or give up**, and neither outcome is one you wanted.

Acknowledge quickly, and state a timeline you will actually meet — **the commonest complaint from reporters is silence**, not disagreement about severity, and silence is what converts a cooperative reporter into an adversarial one.

Fix it, credit the reporter, and publish an advisory with enough detail for your users to assess their own exposure (CSCE 701 Module 08 §4's advisory response depends on somebody having written one).

And never threaten a good-faith reporter — **which has repeatedly converted a quiet private report into a public incident with far worse coverage than the vulnerability warranted**, and is the single worst available response. It is also, on the record, ineffective at suppressing anything.

- **Report privately first**, with enough detail to reproduce reliably, and a clear statement of the impact you established (Module 01 §3's honest form).
- **Offer a reasonable timeline** — ninety days is the common norm — and **say what you will do when it expires**, so the maintainer can plan rather than guess.
- **Coordinate publication**, so that users have a fix available at the moment they learn there is a problem — which is the entire point of coordination.

- **And never test against systems you do not own** (Module 01 §4) — **which is precisely where good intentions become unlawful**, and where a finding you cannot lawfully have obtained becomes a finding you cannot report.
- **This is Project 2's professional requirement, and it is graded** as an obligation rather than a courtesy. **'Report privately, with a timeline, and coordinate publication' is the whole of responsible disclosure** — and **the timeline is what makes it responsible rather than indefinite**, since a report with no deadline can be ignored forever at the users' expense.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Google — Building Secure and Reliable Systems, the development chapters (free PDF)	§1 and §2, free in full — and the safe-by-construction argument is made better here than anywhere else.
Microsoft SDL, and the SAMM maturity model (free)	§1 as published frameworks — useful for structure, and read §1's callout before adopting wholesale.
security.txt (RFC 9116) (free)	§4's first step , which takes ten minutes and is still missing from most sites.
CERT Guide to Coordinated Vulnerability Disclosure (free)	§4 from both sides — the obligations of reporter and maintainer, with the hard cases discussed.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Map your own project's practices** onto Part 1's seven stages, and find the empty rows.
- 2 **Determine whether you threat model at all**, and what replaced it if not.
- 3 **Distinguish three practices that move work left** from three that move the guarantee left.
- 4 **Find the defect class you keep fixing**, from your own history.
- 5 **Design the interface** that would make it unexpressible.
- 6 **Make the unsafe path harder to reach**, and add the lint rule.
- 7 **Time your slowest security gate** and decide whether it will be skipped.
- 8 **Add an auditable override** to a gate that lacks one.
- 9 **Publish a security.txt** for a project you maintain.
- 10 **Write the report** you would send for a defect you found, in full.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Give the seven stages and the practice at each.
- 2 Which row has no automated substitute, and why is it skipped?
- 3 What is right and wrong about 'shift left'?
- 4 State the honest version in one sentence.
- 5 Why do defaults beat training, and give four examples.
- 6 Give the four steps to build a safe-by-default layer.
- 7 Which step decides the outcome, and what kind of requirement is it?
- 8 Give five properties of a gate that survives, and why the override matters.
- 9 Give four things to do when receiving a report.
- 10 Give the three elements of responsible disclosure.